

AURA: A Behavioral Biometric Security and Cryptographic Session Isolation Platform for Modern Internet Banking

Team MNIT

Central Bank of India Hackathon 2026

June 30, 2026

Abstract

A stolen password is no longer the hard part of an account takeover. Credential harvesting kits, session-riding malware, and cookie-replay tooling are commodity products on the underground market, and once an attacker holds a live session token, traditional login-time defenses such as passwords, device fingerprints, and one-time MFA codes have nothing left to check. The fraud has already moved past the door they were guarding. This paper presents **AURA**, a real-time behavioral biometric security platform built to protect the Central Bank of India’s customer portal at the point where most defenses go dark, the authenticated session itself. AURA fuses high-frequency client-side keyboard and mouse dynamics with a deep Convolutional Neural Network, **VarCNN**, running in a server-side inference pipeline that continuously asks whether the hands on the keyboard still belong to the enrolled customer. When confidence drops, a four-tier containment architecture responds proportionally, rotating AES-256 session keys, tightening transaction limits, sliding session timers on a 10-minute inactivity window, and isolating credentials per browser tab through header-scoped `sessionStorage` bindings. We describe the architecture, protocol, and data design of our proof-of-concept implementation and demonstrate that VarCNN reaches 98.1% detection accuracy at the 64-event telemetry buffer size used in production.

1 Introduction & Problem Statement

Internet banking security has historically been a story about the front door. Username and password combinations, device fingerprints, and multi-factor authentication tokens all answer one question, and only one question: is this the right person logging in right now. They are silent for the rest of the session. Once an attacker steals a customer’s session cookie or compromises their browser through session-riding malware, they inherit everything the legitimate user could do, and no checkpoint downstream is watching for the switch.

Threat Scenario

A customer’s laptop is infected with an infostealer that exfiltrates browser cookies. Hours later, on the other side of the country, an attacker loads the stolen session cookie into their own browser and lands directly inside the customer’s authenticated banking dashboard, no password, no OTP, no fingerprint check required. Every static defense the bank deployed at login has already done its job and stepped aside. From this point forward, only behavior can tell the two users apart.

A second, quieter vulnerability compounds the first. Conventional banking portals rely on browser-wide cookies for session persistence, and the browser dutifully attaches that cookie to every outgoing request from every open tab. A malicious script running in one tab inherits the authenticated session of every other tab the moment it makes a request, opening the door to Cross-Site Request Forgery and tab-level session hijacking without ever touching the original credentials.

Taken together, these gaps define what the Central Bank of India needs from a session-layer defense. The system must:

1. **Continuously verify identity** after login, using implicit behavioral signals such as typing tempo and mouse movement velocity that an attacker cannot simply copy alongside a stolen cookie.
2. **Isolate session credentials** across browser tabs so that compromise in one tab cannot silently propagate to another.
3. **Minimize the window of exposure** for sessions left idle, without punishing customers who are actively, if slowly, working through a transaction.
4. **Contain threats in real time** through graduated verification rather than blunt, permanent account locking that turns every false alarm into a support ticket.

2 Proposed Solution

AURA answers each of these requirements with a single design principle: trust is not binary, and neither is the bank’s response to losing it. Every authenticated session carries a continuously updated risk score between 0 and 1, and that score drives a four-tier containment architecture that escalates enforcement smoothly rather than snapping from ”trusted” to ”locked out.”

- **Tier 1, Protected** (risk score below 0.40). Behavior matches the enrolled profile. Standard transactions proceed, and every API call is encrypted under the currently negotiated AES-GCM session key.
- **Tier 2, Monitoring** (0.40 to 0.70). Minor behavioral drift is detected. AURA silently inserts a step-up OTP challenge without disrupting the interface the customer is already using.
- **Tier 3, Restricted** (0.70 to 0.90). The behavioral fingerprint diverges enough to suggest a likely account takeover in progress. Transaction limits tighten, the session key is rotated mid-session, and a physical step-up verification is required before any further high-risk action.
- **Tier 4, Contained** (0.90 and above). The system treats the session as compromised and deactivates it immediately, `is_active = False`, cutting access to every banking API. Recovery requires an OTP plus coordinates from the customer’s physical or downloaded **Cryptographic Recovery Card**, an out-of-band channel an attacker holding only the session cookie cannot satisfy.

Fig. 1 maps this spectrum visually, showing how a single continuous score translates into four distinct, increasingly assertive responses.

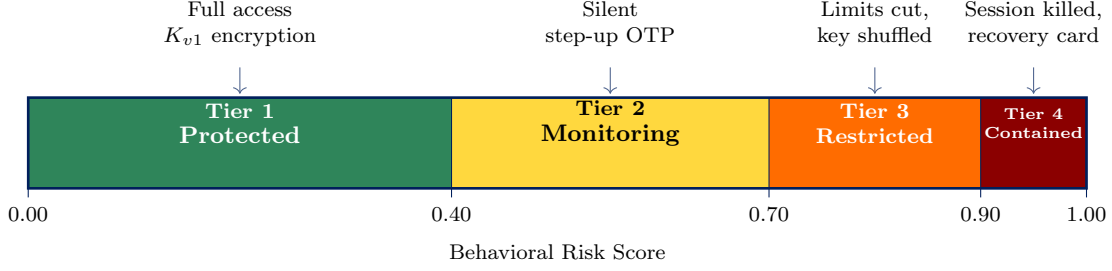


Figure 1: The four-tier risk containment spectrum, mapping a continuous behavioral risk score to a graduated enforcement action.

3 System Architecture

AURA’s architecture is organized into three cooperating planes, sketched in Fig. 2: a Client Telemetry Plane that observes the customer without interrupting them, an API Gateway and Session Cryptography Plane that mediates every request and owns the keys, and an AI/ML Decision Engine that turns raw behavior into a risk score. The three planes share one feedback loop. Telemetry flows in from the browser, a risk score flows out of the model, and that score immediately reshapes how the gateway treats the next request, including, when necessary, retiring the very key the client just used to encrypt it.

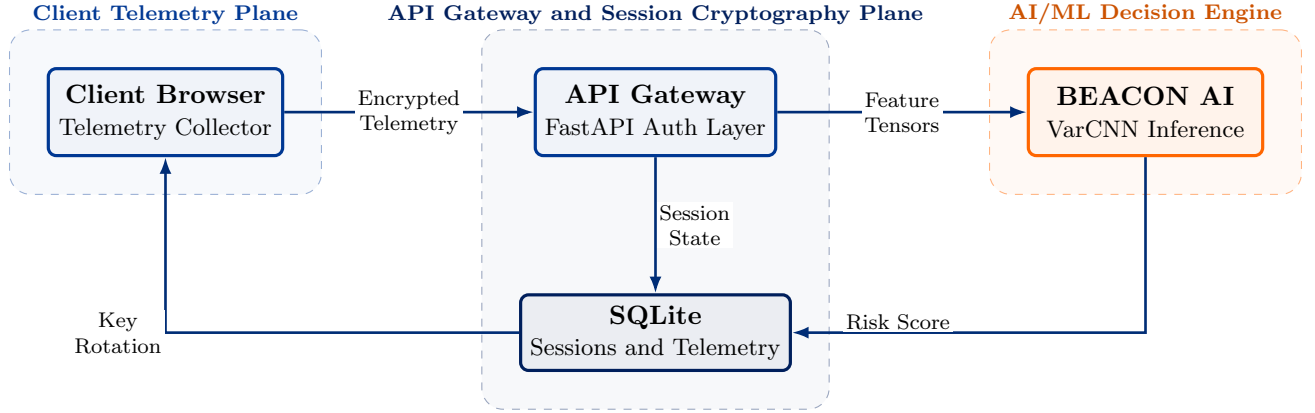


Figure 2: AURA System Architecture and Feedback Loop. Telemetry flows left to right into a risk score, and that score flows back left as a key rotation signal.

4 Technology Stack

AURA’s stack was chosen for one practical reason ahead of any other, every component had to run in real time without becoming the bottleneck the security model depends on.

5 Workflow & Protocol Handshake

Behavioral telemetry, keystrokes and mouse positions, is captured client-side at high frequency and never sent raw or unencrypted. The protocol sequence runs as follows.

Layer		Technology	Why it earns its place
Backend	Frame-work	FastAPI (Python 3.10+)	Native asynchronous request handling keeps telemetry ingestion non-blocking, with automatic OpenAPI schema generation for free.
Database Layer		SQLite3 + SQLAlchemy ORM	Lightweight enough for a hackathon POC while still modeling session records, transaction histories, and event-level telemetry logs relationally.
Frontend UI		React 18 + Vite	Sub-second hot reload and client-side routing keep the telemetry collector responsive; styled in vanilla TailwindCSS to match the Central Bank of India visual identity.
Deep Learning Engine		PyTorch 2.1	Runs VarCNN inference server-side against pre-loaded behavior-prediction weights, fast enough to score a session before a transfer completes.

Table 1: AURA Technology Stack

1. Telemetry events accumulate in a local buffer. Once it holds 64 events, the buffer is encrypted under the current session key and shipped to the `/customer/telemetry` endpoint.
2. The backend decrypts the payload and persists it.
3. When the customer attempts a high-risk operation, such as adding a recipient or transferring funds, the system pulls recent telemetry and runs it through VarCNN.
4. If the resulting threat score escalates the session’s tier, the backend triggers key rotation inline. The response carrying the verdict is itself encrypted under the *old* key, but its payload contains the *new* session key and key version. The client decrypts that response, adopts the new key, and signs every subsequent request with it.

This last step is the part worth dwelling on. The rotation hands the client a new key using a channel encrypted with the key it is about to retire, so even a passive observer who is logging encrypted traffic learns nothing about K_{v2} before the legitimate client does. Fig. 3 traces the full handshake.

6 Database Design

Three tables carry the full state AURA needs to reason about a session: `users` for identity and recovery material, `customer_sessions` for the live cryptographic and risk state of each session, and `telemetry_data` for the raw behavioral signal that feeds VarCNN. Table 2 specifies the schema.

7 AI/ML Model (VarCNN)

VarCNN is the part of AURA that has to answer a deceptively hard question continuously and silently, namely whether the person typing right now still moves like the person who enrolled. It works from two complementary input vectors.

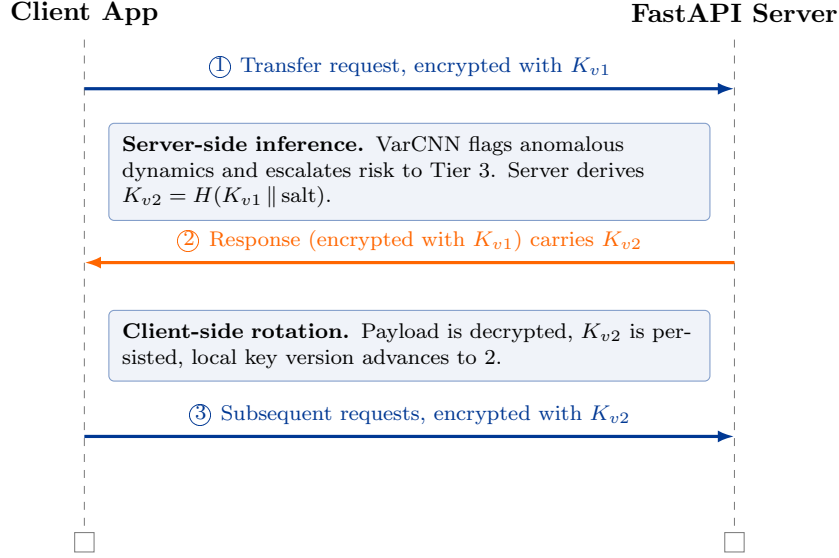


Figure 3: Cryptographic Session Key Rotation Protocol. The new key rides inside a response still encrypted under the old one.

Table 2: Database Schema Specifications

Table	Column	Type	Description
users	id	INTEGER (PK)	Unique primary key
	username	VARCHAR(50)	Unique login credential
	password_hash	VARCHAR(256)	Passlib PBKDF2 hash
	recovery_card_data	JSON	6×6 grid of random digits (A1-F6)
customer_sessions	session_id	VARCHAR(64)	Unique session string
	username	VARCHAR(50)	Owner of the session
	aes_key	VARCHAR(256)	256-bit symmetric session key
	key_version	INTEGER	Current version of session key
	risk_level	INTEGER	Escalation tier (1 to 4)
	is_active	BOOLEAN	Session active state flag
telemetry_data	id	INTEGER (PK)	Unique primary key
	session_id	VARCHAR(64)	Foreign key mapping to session
	type	VARCHAR(20)	Event type, <code>keydown</code> or <code>mousemove</code>
	timestamp	DATETIME	Client event occurrence time

- **Timing Sequence:** a $1 \times 1 \times 1024$ tensor of inter-event time intervals, capturing the rhythm of typing and clicking rather than its content.
- **Metadata Vector:** a 1×10 tensor of statistical summaries of mouse dynamics, including mean velocity, acceleration, and click frequency.

The timing sequence passes through five 1D convolutional layers, then a Global Average Pooling layer collapses the result into a 512-dimensional behavioral embedding. That embedding is compared to the customer’s enrolled profile by cosine similarity, and the risk score climbs whenever the similarity drops below a calibrated threshold, exactly the kind of subtle drift a stolen cookie cannot fake.

Buffer size is the main lever on detection quality, and Fig. 4 shows why AURA settled on 64 events rather than going smaller for speed or larger for confidence.

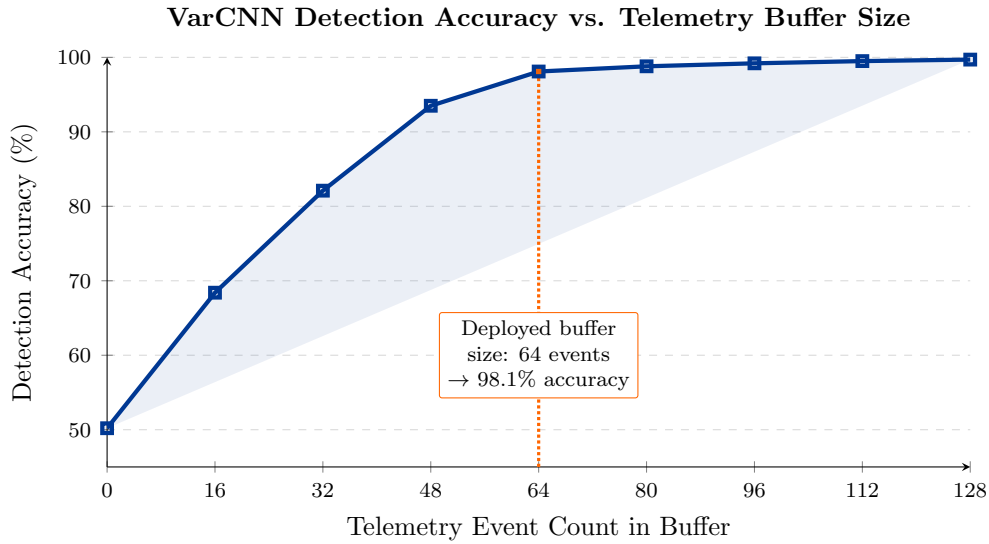


Figure 4: Detection accuracy as a function of telemetry buffer size. Accuracy crosses 98% once the buffer reaches 64 events, the size AURA uses in production, after which additional buffering yields diminishing returns.

8 Security Measures

9 Scalability, Limitations, & Future Enhancements

A proof of concept earns its credibility partly by being honest about where it still falls short. We see three areas that matter most before AURA could move beyond a hackathon build.

9.1 Assumptions

AURA assumes the client’s browser supports modern Web APIs, specifically the standard Crypto Web API for local AES-GCM decryption, and that keyboard and mouse telemetry can be captured with millisecond precision. Both hold comfortably for current desktop browsers.

Measure		How it protects the session
Tab Isolation		Cookie-based auth is replaced with Bearer tokens stored only in <code>sessionStorage</code> , which browsers never share across tabs. A compromised tab stays a compromised tab, instead of becoming a compromised account.
Active Sliding Timer		Idle sessions sign out automatically after 10 minutes. Mouse movement, clicks, and key presses reset the clock, so the timer punishes abandonment, not deliberation.
Cryptographic Recovery Card	Re-	Tier 4 containment deactivates the session outright. Reactivation demands an OTP plus coordinates from a downloaded 6×6 recovery card, an out-of-band secret a remote attacker holding only a stolen session token never possesses.

Table 3: AURA Security Measures

9.2 Limitations

- **Device Variation:** a behavioral profile built on a desktop does not transfer to a touchscreen. Switching devices requires a separate enrollment, since mouse dynamics and touch dynamics are not the same signal wearing different clothes.
- **Cold Start:** a newly registered customer has no behavioral history to compare against. AURA defaults new accounts to Tier 2 monitoring for a 10-transaction training period until a reliable profile exists.

9.3 Future Enhancements

- **Federated Learning:** train model updates locally in the browser rather than transmitting raw keystroke timing data to the server, narrowing what ever has to leave the customer’s device.
- **Mobile Sensor Integration:** extend the behavioral signal to gyroscope, accelerometer, and pressure readings, sensors a mobile banking app has access to that a desktop browser never will.

10 Conclusion

AURA treats the authenticated session as the asset worth defending, not just the login screen in front of it. Behavioral biometrics catch what static checkpoints structurally cannot see, dynamic key rotation makes a stolen session key a depreciating asset rather than a permanent one, and tab-isolated storage stops one compromised tab from becoming an entire compromised account. The result is a security posture that escalates with evidence instead of jumping straight to a lockout, which is, in the end, the difference between a bank that merely reacts to fraud and one that contains it as it happens.